COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE212 - Automating Server Management	**SECTION:**	CPE31S2
DATE PERFORMED:	8/27/2026	**DATE SUBMITTED:**	8/29/2026
NAME:	Momo, Borge Rudolf A.	**INSTRUCTOR:**	Engr. Quejado

ACTIVITY NO. 5.1 **Implement Ansible roles in playbooks**

1. PROCEDURE OUTPUT

Task 1: Use when command for different distributions

1. In the local machine, make sure you are in the local repository directory (*CPE232_yourname*). Issue the command git pull. When prompted, enter the correct passphrase or password. Describe what happened when you issued this command. Did something happen? Why?

```
momo@workstation:~/CPE212_MOMO$ git pull
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 6 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 883 bytes | 441.00 KiB/s, done.
From github.com:BorgeMomo/CPE212_MOMO
   a074d29..a317a2a  main       -> origin/main
Updating a074d29..a317a2a
Fast-forward
 ansible.cfg        |  8 ++++++++
 install_apache.yml | 16 ++++++++++++++++
 inventory.ini      |  6 ++++++
 new.yml            |  9 +++++++++
 4 files changed, 39 insertions(+)
 create mode 100644 ansible.cfg
 create mode 100644 install_apache.yml
 create mode 100644 inventory.ini
 create mode 100644 new.yml
momo@workstation:~/CPE212_MOMO$
```

- It pulled the contents of the ones I uploaded in the repository, which were from the previous hands-on-activity.

2. Edit the inventory file and add the IP address of the Centos VM. Issue the command we used to execute the playbook (the one we used in the last activity): *ansible-playbook --ask-become-pass install_apache.yml*. After executing this command, you may notice that it did not become successful in the Centos VM. You can see that the Centos VM has failed=1. Only the two remote servers have been changed. The reason is that Centos VM does not support "apt" as the package manager. The default package manager for Centos is "yum."

Editing the inventory.ini:

```
 GNU nano 7.2                                  inventory.ini *
[servers]
server1 ansible_host=192.168.56.103 ansible_user=momo2
server2 ansible_host=192.168.56.104 ansible_user=momo3
centos ansible_host=192.168.56.116 ansible_user=momo
```

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]
ok: [centos]
ok: [server2]

TASK [update repository index] *************************************************
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [centos]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[E
'apt-get'", "rc": 2, "stderr": "", "stderr_lines": [], "stdout": "", "stdout_lines
^C [ERROR]: User interrupted execution
momo@workstation:~/CPE212_MOMO$
```

- We can see that CentOS failed.

3. Edit the *install_apache.yml* file and insert the lines shown below.

```
 GNU nano 7.2                                  install_apache.yml *
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name: apache2
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
    when: ansible_distribution == "Ubuntu"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server1]
ok: [server2]
ok: [centos]

TASK [update repository index] *************************************************************************************
skipping: [centos]
changed: [server1]
changed: [server2]

TASK [install apache2 package] *************************************************************************************
skipping: [centos]
changed: [server1]
changed: [server2]

TASK [add PHP support for apache] **********************************************************************************
skipping: [centos]
changed: [server2]
changed: [server1]

PLAY RECAP *********************************************************************************************************
centos                     : ok=1    changed=0    unreachable=0    failed=0    skipped=3    rescued=0    ignored=0
server1                    : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2                    : ok=4    changed=3    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

- After running the command, the tasks applied to the Ubuntu manage nodes but it skipped for the CentOS.

If you have a mix of Debian and Ubuntu servers, you can change the configuration of your playbook like this.

- name: update repository index
  apt:
    update_cache: yes
  when: ansible_distribution in [“Debian”, “Ubuntu]

Note: This will work also if you try. Notice the changes are highlighted.

4. Edit the *install_apache.yml* file and insert the lines shown below.

```
- name: update repository index
  dnf:
    update_cache: yes
  when: ansible_distribution == "CentOS"

- name: install apache2 package
  dnf:
    name: httpd
    state: latest
  when: ansible_distribution == "CentOS"

- name: add PHP support for apache
  dnf:
    name: php
    state: latest
  when: ansible_distribution == "CentOS"
```

Make sure to save and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]
ok: [centos]
ok: [server2]

TASK [update repository index] *************************************************
skipping: [centos]
changed: [server2]
changed: [server1]

TASK [install apache2 package] *************************************************
skipping: [centos]
ok: [server2]
ok: [server1]

TASK [add PHP support for apache] **********************************************
skipping: [centos]
ok: [server1]
ok: [server2]

TASK [update repository index] *************************************************
skipping: [server1]
skipping: [server2]
ok: [centos]
```

```
TASK [install apache2 package] *************************************************
skipping: [server1]
skipping: [server2]
changed: [centos]

TASK [add PHP support for apache] **********************************************
skipping: [server1]
skipping: [server2]
changed: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=4    changed=2    unreachable=0    failed=0    skipped=3
server1                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
server2                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
```

- We can see that the first tasks which were set to Ubuntu, skipped the CentOS manage node. On the other hand, The tasks I have recently uploaded which were meant for CentOS, worked for CentOS manage node and skipped the Ubuntu manage nodes.

5. To verify the installations, go to CentOS VM and type its IP address on the browser. Was it successful? The answer is no. It's because the httpd service or the Apache HTTP server in the CentOS is not yet active. Thus, you need to activate it first.

   5.1 To activate, go to the CentOS VM terminal and enter the following:

   systemctl status httpd

   The result of this command tells you that the service is inactive.

```
[momo@centos ~]$ systemctl status httpd
○ httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/httpd.service.d
             └─php-fpm.conf
     Active: inactive (dead)
       Docs: man:httpd.service(8)
```

   5.2 Issue the following command to start the service:

   sudo systemctl start httpd

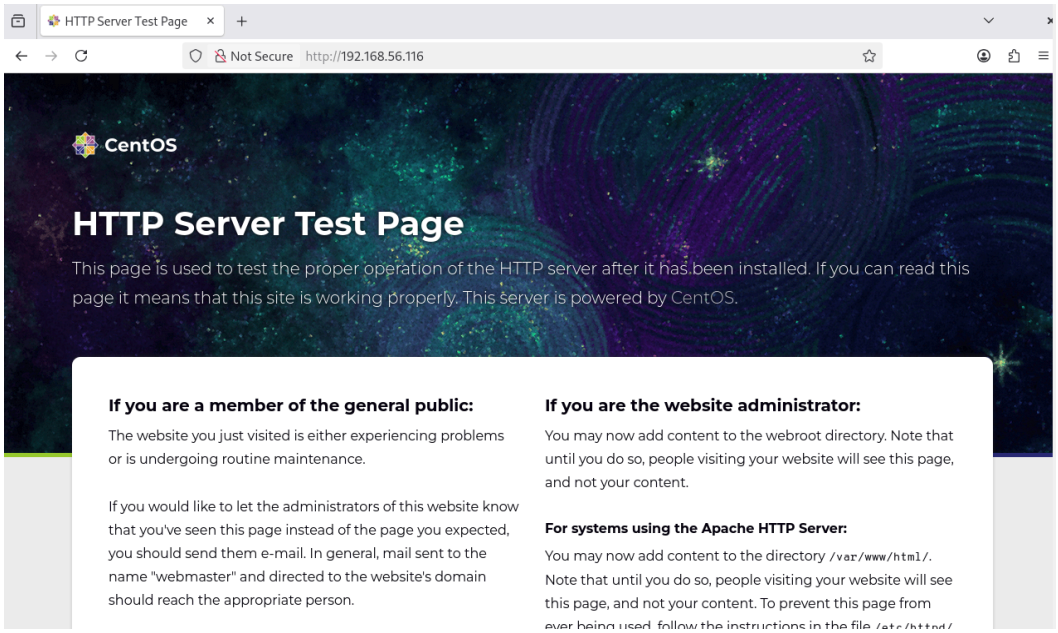
   (When prompted, enter the sudo password)

   sudo firewall-cmd --add-port=80/tcp

   (The result should be a success)

```
[momo@centos ~]$ sudo systemctl start httpd
[sudo] password for momo:
[momo@centos ~]$ sudo firewall-cmd --add-port=80/tcp
success
[momo@centos ~]$
```

   5.3 To verify the service is already running, go to CentOS VM and type its IP address on the browser. Was it successful? (Screenshot the browser)



- We can see that it was successful.

Task 2: Refactoring playbook

This time, we want to make sure that our playbook is efficient and that the codes are easier to read. This will also makes run ansible more quickly if it has to execute fewer tasks to do the same thing.

1. Edit the playbook *install_apache.yml*. Currently, we have three tasks targeting our Ubuntu machines and 3 tasks targeting our CentOS machine. Right now, we try to consolidate some tasks that are typically the same. For example, we can consolidate two plays that install packages. We can do that by creating a list of installation packages as shown below:

```
  GNU nano 7.2                                    install_apache.yml
---
- hosts: all
  become: true
  tasks:

  - name: update repository index
    apt:
      update_cache: yes
    when: ansible_distribution == "Ubuntu"

  - name: install apache2 package
    apt:
      name:
        - apache2
        - libapache2-mod-php
      state: latest
    when: ansible_distribution == "Ubuntu"

  - name: add PHP support for apache
    apt:
      name: libapache2-mod-php
    when: ansible_distribution == "Ubuntu"

  - name: update repository index
    dnf:
      update_cache: yes
    when: ansible_distribution == "CentOS"
```

```
  - name: install apache2 package
    dnf:
      name:
        - httpd
        - php
      state: latest
    when: ansible_distribution == "CentOS"

  - name: add PHP support for apache
    dnf:
      name: php
      state: latest
    when: ansible_distribution == "CentOS"
```

Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [server2]
ok: [server1]
ok: [centos]

TASK [update repository index] *************************************************
skipping: [centos]
changed: [server2]
changed: [server1]

TASK [install apache2 package] *************************************************
skipping: [centos]
ok: [server1]
ok: [server2]

TASK [add PHP support for apache] **********************************************
skipping: [centos]
ok: [server2]
ok: [server1]

TASK [update repository index] *************************************************
skipping: [server1]
skipping: [server2]
ok: [centos]
```

```
TASK [install apache2 package] *************************************************
skipping: [server1]
skipping: [server2]
ok: [centos]

TASK [add PHP support for apache] **********************************************
skipping: [server1]
skipping: [server2]
ok: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=4    changed=0    unreachable=0    failed=0    skipped=3
server1                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
server2                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
```

- The playbook successfully installed the required packages on both Ubuntu and CentOS after consolidating the package installation tasks.

2. Edit the playbook *install_apache.yml* again. In task 2.1, we consolidated the plays into one play. This time we can actually consolidated everything in just 2 plays. This can be done by removing the update repository play and putting the command *update_cache: yes* below the command *state: latest*. See below for reference:
   Make sure to save the file and exit.

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************

TASK [Gathering Facts] *********************************************************
ok: [server1]
ok: [centos]
ok: [server2]

TASK [update repository index] *************************************************
skipping: [centos]
changed: [server1]
changed: [server2]

TASK [install apache2 package] *************************************************
skipping: [centos]
ok: [server2]
ok: [server1]

TASK [add PHP support for apache] **********************************************
skipping: [centos]
ok: [server1]
ok: [server2]

TASK [update repository index] *************************************************
skipping: [server1]
skipping: [server2]
ok: [centos]
```

```
TASK [install apache2 package] *************************************************
skipping: [server1]
skipping: [server2]
ok: [centos]

TASK [add PHP support for apache] **********************************************
skipping: [server1]
skipping: [server2]
ok: [centos]

PLAY RECAP *********************************************************************
centos                     : ok=4    changed=0    unreachable=0    failed=0    skipped=3
server1                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
server2                    : ok=4    changed=1    unreachable=0    failed=0    skipped=3
```

- The playbook successfully ran with fewer tasks by combining the repository update with the package installation.

3. Finally, we can consolidate these 2 plays in just 1 play. This can be done by declaring variables that will represent the packages that we want to install. Basically, the apache_package and php_package are variables. The names are arbitrary, which means we can choose different names. We also take out the line when: ansible_distribution. Edit the playbook *install_apache.yml* again and make sure to follow the below image. Make sure to save the file and exit.

```
  GNU nano 7.2
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php
    apt:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************************************

TASK [Gathering Facts] *********************************************************************************************
ok: [server2]
ok: [server1]
ok: [centos]

TASK [install apache and php] **************************************************************************************
[WARNING]: Updating cache and auto-installing missing dependency: python3-apt
fatal: [centos]: FAILED! => {"changed": false, "cmd": "apt-get update", "msg": "[Errno 2] No such file or dire
'apt-get'", "rc": 2, "stderr": "", "stderr_lines": [], "stdout": "", "stdout_lines": []}
ok: [server2]
ok: [server1]

PLAY RECAP *********************************************************************************************************
centos                     : ok=1    changed=0    unreachable=0    failed=1    skipped=0    rescued=0    ignor
server1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
server2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignor
```

- After I ran the command, it was not successful for CentOS but successful for the Ubuntu manage nodes.

4. Unfortunately, task 2.3 was not successful. It's because we need to change something in the inventory file so that the variables we declared will be in place. Edit the *inventory* file and follow the below configuration:

```
  GNU nano 7.2                                  inventory.ini
[servers]
server1 ansible_host=192.168.56.115 ansible_user=momo apache_package=apache2 php_package=libapache2-mod-php
server2 ansible_host=192.168.56.114 ansible_user=momo apache_package=apache2 php_package=libapache2-mod-php
centos ansible_host=192.168.56.116 ansible_user=momo apache_package=httpd php_package=php

[all:vars]
ansible_python_interpreter=/usr/bin/python3
```

Make sure to save the *inventory* file and exit.

Finally, we still have one more thing to change in our *install_apache.yml* file. In task 2.3, you may notice that the package is assign as apt, which will not run in CentOS. Replace the *apt* with *package*. Package is a module in ansible that is generic, which is going to use whatever package manager the underlying host or the target server uses. For Ubuntu it will automatically use *apt*, and for CentOS it will automatically use *dnf*. Make sure to save the file and exit. For more details about the ansible package, you may refer to this documentation: ansible.builtin.package – Generic OS package manager — Ansible Documentation

```
---
- hosts: all
  become: true
  tasks:

  - name: install apache and php
    package:
      name:
        - "{{ apache_package }}"
        - "{{ php_package }}"
      state: latest
      update_cache: yes
```

Run *ansible-playbook --ask-become-pass install_apache.yml* and describe the result.

```
momo@workstation:~/CPE212_MOMO$ ansible-playbook --ask-become-pass install_apache.yml
BECOME password:

PLAY [all] *********************************************************************************************************

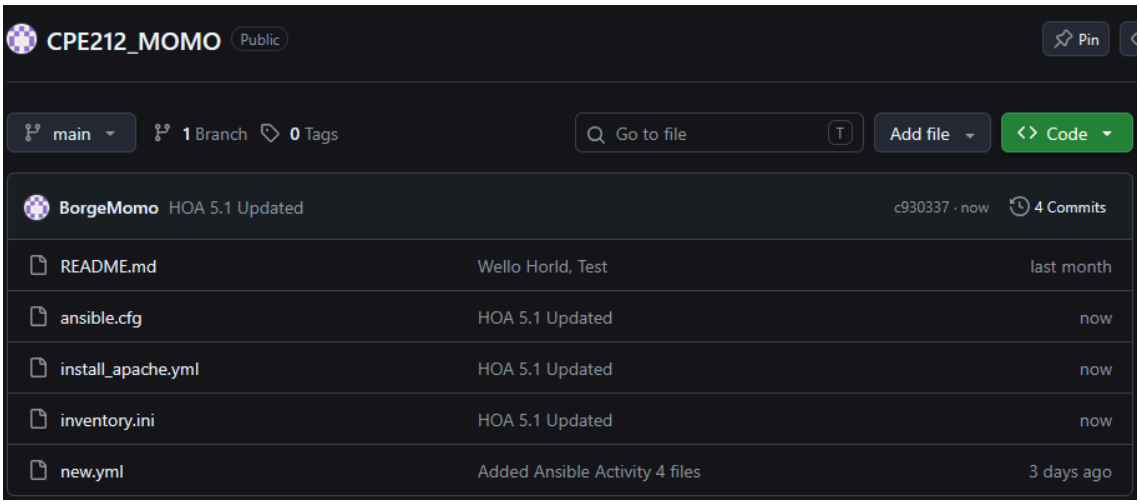
TASK [Gathering Facts] *********************************************************************************************
ok: [server2]
ok: [server1]
ok: [centos]

TASK [install apache and php] **************************************************************************************
ok: [server2]
ok: [server1]
ok: [centos]

PLAY RECAP *********************************************************************************************************
centos                     : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server1                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
server2                    : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

- The playbook successfully installed the Apache and PHP packages on both Ubuntu and CentOS after adding the package variables to the inventory and using the generic package module

. **git commit proof:**



2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Supplementary Activity:

1. Create a playbook that could do the previous tasks in Red Hat OS.

```
 GNU nano 8.7.1                 install_apache_redhat.yml
---
- name: Install Apache and PHP on Red Hat
  hosts: redhat
  become: yes

  tasks:
    - name: Install Apache and PHP
      package:
        name:
          - httpd
          - php
        state: latest

    - name: Start Apache
      service:
        name: httpd
        state: started
        enabled: yes
```

Reflections:

Answer the following:

1. **Why do you think refactoring of playbook codes is important?**
   - I think refactoring playbook codes is important because it makes the code cleaner and easier to read. Another is that it can also reduce the number of tasks needed to do the same work which can make the playbook run faster and make it easier to manage. It also helps avoid repeated codes and makes changes easier in the future.

2. **When do we use the "when" command in the playbook?**
   - We use the "when" command when we want a task to run only under a certain condition. An example would be that we can use it when a task should only run on Ubuntu or Debian. This is useful when different operating systems need different commands or tasks and it helps the playbook choose the correct task for each system.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Understanding of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The tool was only used to understand some commands since most of the commands in the procedure were resulting to an error.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.